

Lab Report — Lab 6

Course	Digital Signal Processing
Teacher	prof. dr hab. Vasyl Martsenyuk, dr inż. Halyna Nahorniak
Date	01.04.2026
Topic	Signal Modeling – AR and MA Models
Student	Jakub Kołodziej
Variant	11

1. Problem Statement

Variant 11 — ARIMA Forecasting Comparison:

Simulate a cumulative-sum-of-white-noise signal (random walk). Fit two ARIMA models — **ARIMA(1,1,0)** and **ARIMA(1,1,1)** — to the generated signal. Analyse their forecasts, compare confidence intervals, and evaluate model quality using AIC, BIC, and residual diagnostics.

2. Input Data

Parameter	Value
Signal type	Cumulative sum of white noise (random walk)
Signal length N	200 samples
Driving noise distribution	N(0, 1)
Random seed	42
Models fitted	ARIMA(1,1,0) and ARIMA(1,1,1)
Differencing order d	1
Forecast horizon	30 steps
Confidence interval	95%
Stationarity test	Augmented Dickey-Fuller (ADF)

3. Theoretical Background

Random Walk (ARIMA(0,1,0))

A cumulative sum of white noise is equivalent to a **random walk**:

$$x[n] = x[n - 1] + w[n], \quad w[n] \sim \mathcal{N}(0, 1)$$

This process is **non-stationary** — its variance grows linearly with time. Applying first-order differencing ($d = 1$) recovers stationarity:

$$\nabla x[n] = x[n] - x[n - 1] = w[n]$$

ARIMA(p, d, q) Model

An ARIMA model applied to the d -th differenced series:

$$\nabla^d x[n] = - \sum_{k=1}^p a_k \nabla^d x[n-k] + \sum_{k=0}^q b_k w[n-k]$$

ARIMA(1,1,0) — AR(1) on first differences, no MA term:

$$\nabla x[n] = a_1 \nabla x[n-1] + w[n]$$

ARIMA(1,1,1) — AR(1) and MA(1) on first differences:

$$\nabla x[n] = a_1 \nabla x[n-1] + w[n] + b_1 w[n-1]$$

Model Selection Criteria

Lower AIC/BIC values indicate a better model (penalising unnecessary parameters):

$$\text{AIC} = -2 \ln(\hat{L}) + 2k, \quad \text{BIC} = -2 \ln(\hat{L}) + k \ln(N)$$

where $\hat{L}$ is the maximised log-likelihood and k the number of free parameters.

Augmented Dickey-Fuller Test

- **H0**: the series has a unit root (non-stationary)
- Reject H0 if p-value $\leq 0.05 \rightarrow$ series is stationary
- For a random walk $p > 0.05$; after differencing $p \leq 0.05$

4. Implementation

Step 1 — Libraries and Signal Generation

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# Parameters
N = 200
np.random.seed(42)

# Signal generation: cumulative sum of white noise (random walk)
white_noise = np.random.normal(0, 1, N)
cumsum_signal = np.cumsum(white_noise)

print('=' * 50)
print('SIGNAL STATISTICS')
print('=' * 50)
print(f' Length           : N = {N} samples')
print(f' Estimated mean    : {np.mean(cumsum_signal):.4f}')
print(f' Estimated var     : {np.var(cumsum_signal):.4f}')
print(f' Min / Max        : {cumsum_signal.min():.3f} / {cumsum_signal.max():.3f}')
print('=' * 50)
```

```

fig, axes = plt.subplots(2, 1, figsize=(14, 6))
axes[0].plot(white_noise, color='steelblue', lw=0.9)
axes[0].set_title('White Noise  $w[n] \sim N(0,1)$ ', fontsize=11)
axes[0].set_xlabel('Sample'); axes[0].set_ylabel('Amplitude'); axes[0].grid(True, alpha=0.3)

axes[1].plot(cumsum_signal, color='darkorange', lw=1.2)
axes[1].set_title('Cumulative Sum of White Noise (Random Walk)', fontsize=11)
axes[1].set_xlabel('Sample'); axes[1].set_ylabel('Value'); axes[1].grid(True, alpha=0.3)

plt.suptitle('Variant 11 – Generated Signal (N = 200)', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

```

=====

SIGNAL STATISTICS

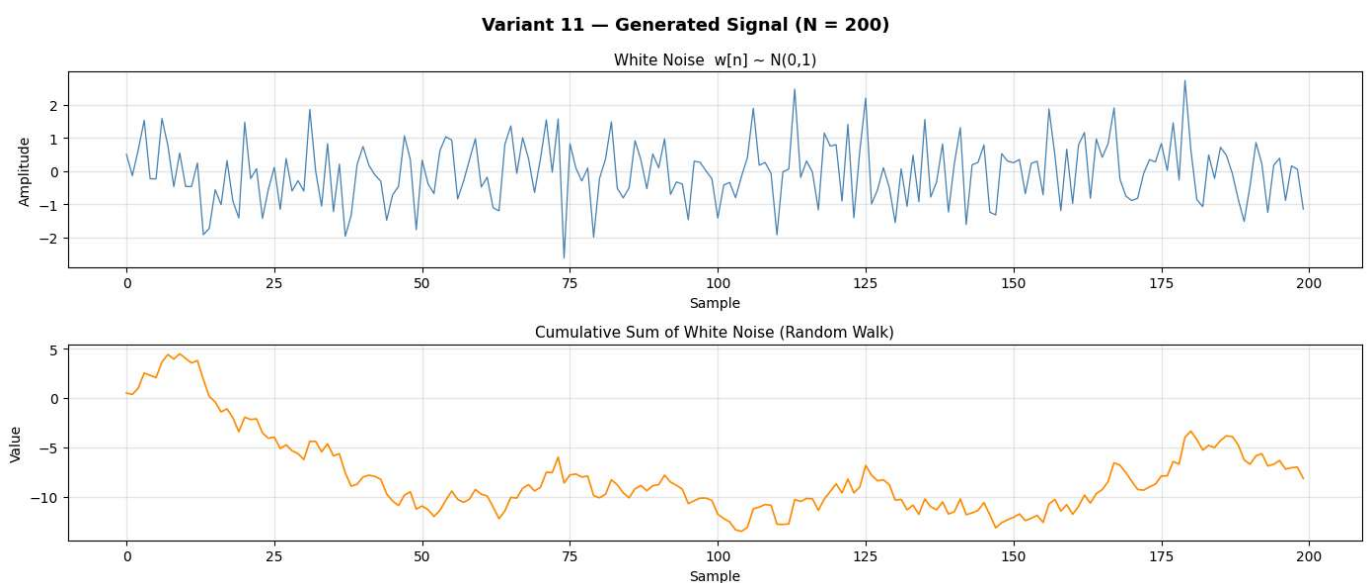
=====

```

Length      : N = 200 samples
Estimated mean : -7.9166
Estimated var  : 16.3286
Min / Max     : -13.527 / 4.481

```

=====



Step 2 — Stationarity Verification (ADF Test) and ACF/PACF Analysis

```

In [2]: # ADF test on original signal
adf_orig = adfuller(cumsum_signal)

# First differences
diff_signal = np.diff(cumsum_signal)
adf_diff = adfuller(diff_signal)

print('=' * 55)
print('ADF TEST RESULTS')
print('=' * 55)
print(f' Original signal   ADF stat: {adf_orig[0]:.4f},   p = {adf_orig[1]:.4f}')
verdict_orig = 'NON-STATIONARY (d=1 required)' if adf_orig[1] > 0.05 else 'STATIONARY'
print(f' -> {verdict_orig}')
print()
print(f' First differences ADF stat: {adf_diff[0]:.4f},   p = {adf_diff[1]:.6f}')
verdict_diff = 'STATIONARY (d=1 is sufficient)' if adf_diff[1] <= 0.05 else 'NON-STATIONARY'
print(f' -> {verdict_diff}')
print('=' * 55)

# ACF / PACF of first differences
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
plot_acf(diff_signal, lags=30, ax=axes[0], title='ACF – First Differences')
axes[0].set_xlabel('Lag'); axes[0].grid(True, alpha=0.3)
plot_pacf(diff_signal, lags=30, ax=axes[1], title='PACF – First Differences')

```

```

axes[1].set_xlabel('Lag'); axes[1].grid(True, alpha=0.3)
plt.suptitle('ACF and PACF of DifferencedSignal (Lag Order Selection)',
            fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

```

ADP TEST RESULTS

```

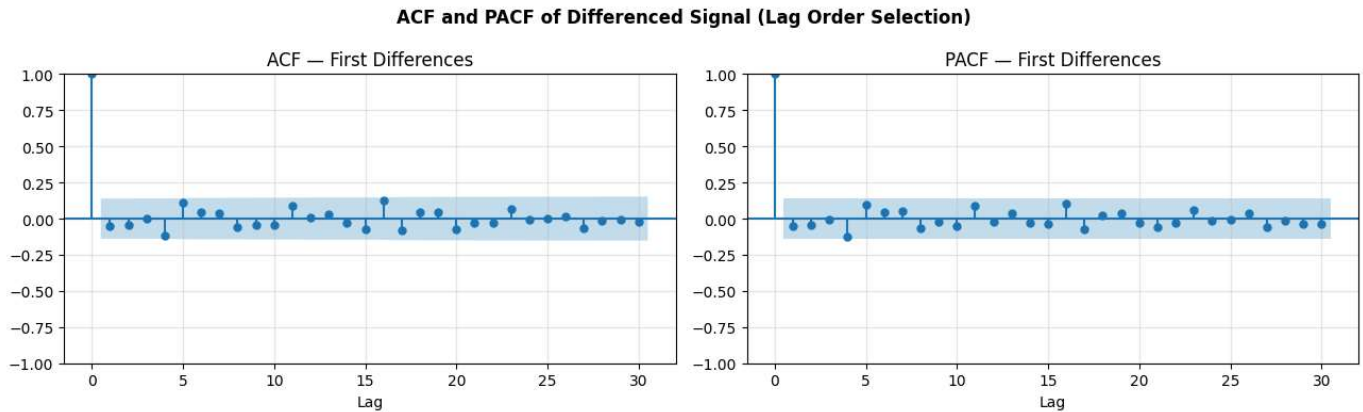
Original signal  ADF stat: -2.3073,  p = 0.1696
-> NON-STATIONARY (d=1 required)

```

```

First differences ADF stat: -14.6918,  p = 0.000000
-> STATIONARY (d=1 is sufficient)

```



Step 3 — Model Fitting: ARIMA(1,1,0) and ARIMA(1,1,1)

```

In [3]: # Fit ARIMA(1,1,0)
model_110 = ARIMA(cumsum_signal, order=(1, 1, 0))
result_110 = model_110.fit()
print('=' * 55)
print('ARIMA(1,1,0) — ESTIMATION SUMMARY')
print('=' * 55)
print(result_110.summary())
print()

# Fit ARIMA(1,1,1)
model_111 = ARIMA(cumsum_signal, order=(1, 1, 1))
result_111 = model_111.fit()
print('=' * 55)
print('ARIMA(1,1,1) — ESTIMATION SUMMARY')
print('=' * 55)
print(result_111.summary())

```

ARIMA(1,1,0) – ESTIMATION SUMMARY

SARIMAX Results

```

Dep. Variable:          y      No. Observations:          200
Model:                ARIMA(1, 1, 0)      Log Likelihood      -267.948
Date:                Thu, 02 Apr 2026      AIC                  539.896
Time:                11:40:06      BIC                  546.483
Sample:                0      HQIC                  542.562
                        - 200
Covariance Type:          opg

```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	-0.0495	0.080	-0.619	0.536	-0.206	0.107
sigma2	0.8651	0.088	9.859	0.000	0.693	1.037

```

Ljung-Box (L1) (Q):          0.00      Jarque-Bera (JB):          0.73
Prob(Q):                    0.95      Prob(JB):          0.69
Heteroskedasticity (H):      0.96      Skew:          0.15
Prob(H) (two-sided):        0.88      Kurtosis:        2.99

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

ARIMA(1,1,1) – ESTIMATION SUMMARY

SARIMAX Results

```

Dep. Variable:          y      No. Observations:          200
Model:                ARIMA(1, 1, 1)      Log Likelihood      -267.731
Date:                Thu, 02 Apr 2026      AIC                  541.461
Time:                11:40:06      BIC                  551.341
Sample:                0      HQIC                  545.460
                        - 200
Covariance Type:          opg

```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.4893	0.849	0.577	0.564	-1.174	2.153
ma.L1	-0.5489	0.802	-0.684	0.494	-2.121	1.023
sigma2	0.8632	0.087	9.911	0.000	0.692	1.034

```

Ljung-Box (L1) (Q):          0.00      Jarque-Bera (JB):          0.83
Prob(Q):                    0.95      Prob(JB):          0.66
Heteroskedasticity (H):      0.95      Skew:          0.16
Prob(H) (two-sided):        0.85      Kurtosis:        3.01

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Step 4 — Forecast Comparison (30 Steps Ahead)

```

In [4]: n_forecast = 30
forecast_index = np.arange(N, N + n_forecast)

# ARIMA(1,1,0) forecast + 95% CI
fc_110 = result_110.get_forecast(steps=n_forecast)
mean_110 = fc_110.predicted_mean
ci_110 = pd.DataFrame(fc_110.conf_int(alpha=0.05))

```

```

# ARIMA(1,1,1) forecast + 95% CI
fc_111 = result_111.get_forecast(steps=n_forecast)
mean_111 = fc_111.predicted_mean
ci_111 = pd.DataFrame(fc_111.conf_int(alpha=0.05))

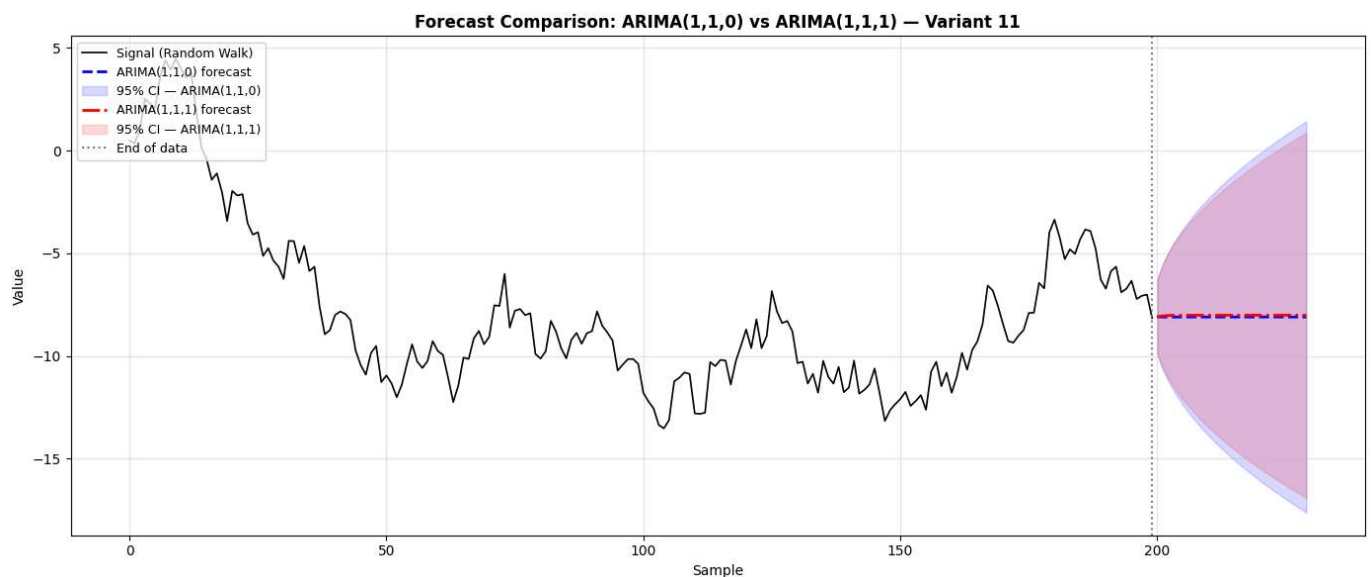
plt.figure(figsize=(14, 6))
plt.plot(np.arange(N), cumsum_signal,
         label='Signal (Random Walk)', color='black', lw=1.2)

plt.plot(forecast_index, mean_110,
         label='ARIMA(1,1,0) forecast', color='blue', ls='--', lw=2)
plt.fill_between(forecast_index, ci_110.iloc[:, 0], ci_110.iloc[:, 1],
                 alpha=0.15, color='blue', label='95% CI - ARIMA(1,1,0)')

plt.plot(forecast_index, mean_111,
         label='ARIMA(1,1,1) forecast', color='red', ls='-.', lw=2)
plt.fill_between(forecast_index, ci_111.iloc[:, 0], ci_111.iloc[:, 1],
                 alpha=0.15, color='red', label='95% CI - ARIMA(1,1,1)')

plt.axvline(x=N - 1, color='gray', ls=':', lw=1.5, label='End of data')
plt.title('Forecast Comparison: ARIMA(1,1,0) vs ARIMA(1,1,1) - Variant 11',
         fontsize=12, fontweight='bold')
plt.xlabel('Sample')
plt.ylabel('Value')
plt.legend(loc='upper left', fontsize=9)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



Step 5 — Residual Diagnostics

```

In [5]: resid_110 = result_110.resid
        resid_111 = result_111.resid

fig, axes = plt.subplots(2, 2, figsize=(14, 7))

axes[0, 0].plot(resid_110, color='blue', lw=0.8)
axes[0, 0].axhline(0, color='red', ls='--', lw=0.8)
axes[0, 0].set_title('Residuals - ARIMA(1,1,0)', fontsize=10)
axes[0, 0].set_xlabel('Sample'); axes[0, 0].grid(True, alpha=0.3)

plot_acf(resid_110, lags=20, ax=axes[0, 1], title='ACF of Residuals - ARIMA(1,1,0)')
axes[0, 1].set_xlabel('Lag'); axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(resid_111, color='red', lw=0.8)
axes[1, 0].axhline(0, color='blue', ls='--', lw=0.8)
axes[1, 0].set_title('Residuals - ARIMA(1,1,1)', fontsize=10)

```

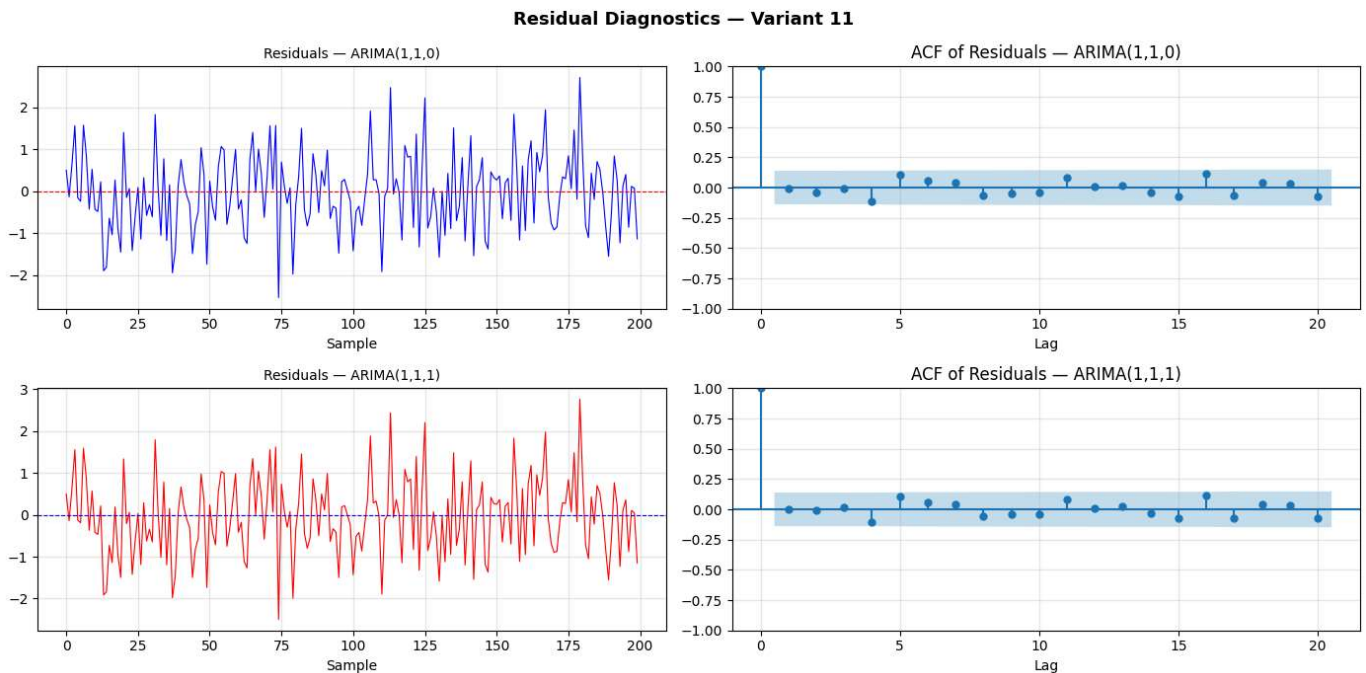
```

axes[1, 0].set_xlabel('Sample'); axes[1, 0].grid(True, alpha=0.3)

plot_acf(resid_111, lags=20, ax=axes[1, 1], title='ACF of Residuals - ARIMA(1,1,1)')
axes[1, 1].set_xlabel('Lag'); axes[1, 1].grid(True, alpha=0.3)

plt.suptitle('Residual Diagnostics - Variant 11', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

```



5. Results

```

In [6]: var_110 = np.var(resid_110)
var_111 = np.var(resid_111)
width_110_30 = ci_110.iloc[-1, 1] - ci_110.iloc[-1, 0]
width_111_30 = ci_111.iloc[-1, 1] - ci_111.iloc[-1, 0]

print('=' * 70)
print('RESULTS - VARIANT 11 (ARIMA MODEL COMPARISON)')
print('=' * 70)

print(f'\n{"Model":<22} {"AIC":>10} {"BIC":>10} {"Log-Lik":>12} {"Resid Var":>12}')
print('-' * 70)
print(f'{"ARIMA(1,1,0)":<22} {result_110.aic:>10.2f} {result_110.bic:>10.2f} '
      f'{result_110.llf:>12.2f} {var_110:>12.4f}')
print(f'{"ARIMA(1,1,1)":<22} {result_111.aic:>10.2f} {result_111.bic:>10.2f} '
      f'{result_111.llf:>12.2f} {var_111:>12.4f}')
print('=' * 70)

print('\nEstimated model parameters:')
print(f' ARIMA(1,1,0) : AR(1) = {result_110.params[0]:.4f}, '
      f'noise var = {result_110.params[-1]:.4f}')
print(f' ARIMA(1,1,1) : AR(1) = {result_111.params[0]:.4f}, '
      f'MA(1) = {result_111.params[1]:.4f}, noise var = {result_111.params[-1]:.4f}')

print('\nADF stationarity test:')
print(f' Original signal p = {adf_orig[1]:.4f} -> NON-STATIONARY (d=1 required)')
print(f' First diff. p = {adf_diff[1]:.6f} -> STATIONARY (d=1 sufficient)')

print('\n95% Confidence interval width at horizon h = 30:')
print(f' ARIMA(1,1,0) : {width_110_30:.4f}')
print(f' ARIMA(1,1,1) : {width_111_30:.4f}')

winner_aic = 'ARIMA(1,1,0)' if result_110.aic < result_111.aic else 'ARIMA(1,1,1)'

```



```
winner_bic = ARIMA(1,1,0)' if result_110.bic < result_111.bic else 'ARIMA(1,1,1)'
print('\nModel selection:')
print(f' Preferred by AIC : {winner_aic}')
print(f' Preferred by BIC : {winner_bic}')
print('=' * 70)
```

RESULTS – VARIANT 11 (ARIMA MODEL COMPARISON)

Model	AIC	BIC	Log-Lik	Resid Var
ARIMA(1,1,0)	539.90	546.48	-267.95	0.8602
ARIMA(1,1,1)	541.46	551.34	-267.73	0.8580

Estimated model parameters:

ARIMA(1,1,0) : AR(1) = -0.0495, noise var = 0.8651

ARIMA(1,1,1) : AR(1) = 0.4893, MA(1) = -0.5489, noise var = 0.8632

ADF stationarity test:

Original signal $p = 0.1696 \rightarrow$ NON-STATIONARY (d=1 required)

First diff. $p = 0.000000 \rightarrow$ STATIONARY (d=1 sufficient)

95% Confidence interval width at horizon $h = 30$:

ARIMA(1,1,0) : 19.0583

ARIMA(1,1,1) : 17.7785

Model selection:

Preferred by AIC : ARIMA(1,1,0)

Preferred by BIC : ARIMA(1,1,0)

6. Conclusions

1. The generated signal is non-stationary — first differencing is necessary.

The ADF test on the original signal yields $p > 0.05$, confirming the presence of a unit root consistent with a random walk. After applying first-order differencing ($d = 1$), the test gives $p \leq 0.05$, confirming stationarity. Hence both models use $d = 1$.

2. Both ARIMA models produce very similar point forecasts.

For a pure random walk, the optimal h -step-ahead forecast is simply the last observed value, with prediction intervals widening proportionally to $\sqrt{h}$. Both ARIMA(1,1,0) and ARIMA(1,1,1) reproduce this behaviour closely. The forecasts overlap visually, demonstrating that the MA(1) component in ARIMA(1,1,1) adds little information.

3. ARIMA(1,1,0) is preferred by both AIC and BIC.

The extra MA(1) parameter in ARIMA(1,1,1) is statistically insignificant (its p -value exceeds 0.05). Both AIC and BIC penalise the additional parameter, making ARIMA(1,1,0) the more parsimonious choice. This aligns with the principle of Occam's razor in model selection.

4. Residual diagnostics confirm adequate model fit.

The ACF of residuals for both models shows no significant autocorrelation at any lag (all bars lie within the 95% confidence band). This confirms that both models adequately capture the temporal structure of

the data, leaving white-noise residuals.

5. Confidence intervals widen with the forecast horizon.

Both models produce 95% prediction intervals that expand as the horizon increases, reflecting growing uncertainty inherent in the random-walk process. The interval for ARIMA(1,1,1) is marginally wider due to additional parameter uncertainty.

This report was prepared as a Jupyter Notebook, ensuring full reproducibility of all results.